

Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Logger.java:

```
class Logger {  
    // Private static instance  
    private static Logger instance;  
    // Private constructor  
    private Logger() {  
        System.out.println("Logger object created.");  
    }  
    // Method to get the single instance  
    public static Logger getInstance() {  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
    // Logging method  
    public void log(String message) {  
        System.out.println("LOG: " + message);  
    }  
}
```

TestLogger.java:

```
class TestLogger {  
    public static void main(String[] args) {
```

```

Logger logger1 = Logger.getInstance();
logger1.log("First message");
Logger logger2 = Logger.getInstance();
logger2.log("Second message");
// Check whether both references are same
if (logger1 == logger2) {
    System.out.println("Only one Logger instance exists.");
} else {
    System.out.println("Multiple instances created.");
}
}
}

```

Output:

```

C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\SingletonPatternExample>javac Logger.java
C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\SingletonPatternExample>javac TestLogger.java
C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\SingletonPatternExample>java TestLogger
Logger object created.
LOG: First message
LOG: Second message
Only one Logger instance exists.

C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\SingletonPatternExample>

```

Exercise 2: Implementing the Factory Method Pattern

Document.java:

```

interface Document {
    void open();
}

```

```
}
```

WordDocument.java:

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Opening Word Document.");  
    }  
}
```

PdfDocument.java:

```
class PdfDocument implements Document {  
    public void open() {  
        System.out.println("Opening PDF Document.");  
    }  
}
```

ExcelDocument.java:

```
class ExcelDocument implements Document {  
    public void open() {  
        System.out.println("Opening Excel Document.");  
    }  
}
```

DocumentFactory.java:

```
abstract class DocumentFactory {  
    public abstract Document createDocument();  
}  
  
class WordFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```
}  
}
```

WordFactory.java:

```
class WordFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

PdfFactory.java:

```
class PdfFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

ExcelFactory.java:

```
class ExcelFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new ExcelDocument();  
    }  
}
```

FactoryTest.java:

```
class FactoryTest {  
    public static void main(String[] args) {  
        DocumentFactory factory;  
        factory = new WordFactory();  
        Document doc1 = factory.createDocument();  
    }  
}
```

```
doc1.open();  
factory = new PdfFactory();  
Document doc2 = factory.createDocument();  
doc2.open();  
factory = new ExcelFactory();  
Document doc3 = factory.createDocument();  
doc3.open();  
}  
}
```

Output:

```
C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\FactoryMethodPatternExample>javac *.java  
C:\Users\HP\Desktop\Deepskilling\Engineering Concepts\Design Patterns and Principles\FactoryMethodPatternExample>java FactoryTest  
Opening Word Document.  
Opening PDF Document.  
Opening Excel Document.
```